Memoris OS Architecture And Tech Stack Decision

Architecture and technology decision record for the Memoris OS hackathon demo.

Demo purpose Upload this file, ask questions, verify evidence.	**Security pattern** Tenant scope and role checks happen before AI context.	**Business value** Teams recover decisions without reading every document.

Purpose

Memoris OS was built as a work and productivity platform for organizations that need a trusted memory layer. The goal is to capture meetings, documents, decisions, action items, and timeline events, then let users ask natural language questions with evidence-backed answers.

The project is intentionally backend-first. A real enterprise memory product needs authentication, organization-level isolation, role-based access control, document ingestion, retrieval, and secure AI orchestration. A frontend-only app would not prove those engineering decisions.

Chosen Architecture

Memoris OS uses a React TypeScript frontend, a Java Spring Boot backend, Neon PostgreSQL with pgvector, JWT authentication, and Gemini/OpenAI-ready AI services.

The live demo is deployed with Vercel for the frontend and AWS EC2 with Nginx for the backend. Neon PostgreSQL stores structured organization data and vector embeddings.

Why React, TypeScript, Vite, And Tailwind

React and TypeScript make the dashboard interactive, maintainable, and type-safe. Vite keeps local development and production builds fast. Tailwind makes it possible to build a polished UI quickly while still keeping the design system consistent.

This stack supports the demo workflows: landing page, login, dashboard, Knowledge upload, Timeline, Search, Ask Memoris, and evidence viewer.

Why Java And Spring Boot

Spring Boot was chosen because Memoris OS needs a serious backend. It provides strong support for REST APIs, validation, dependency injection, security, database access, and production deployment.

For an SDE-focused project, Java and Spring Boot show backend engineering depth. The system is not only a UI mockup. It has real authentication, RBAC, database migrations, API endpoints, document processing, and background-ready deployment.

Why PostgreSQL And pgvector

PostgreSQL is the system of record because it supports reliable relational modeling for users, organizations, meetings, decisions, action items, documents, and timeline events.

pgvector lets the same database also store document embeddings for semantic search. This keeps the MVP simpler than running a separate vector database while still supporting real retrieval-augmented generation.

CockroachDB was considered for distributed SQL, but it is not necessary for the first version. PostgreSQL is easier to operate, faster to demo, and strong enough for the current architecture. CockroachDB remains a future option if multi-region distributed SQL becomes a product requirement.

Why Neon

Neon provides managed PostgreSQL with pgvector support, which makes it a good fit for the hackathon. It avoids the operational risk of managing a database directly on EC2 while still giving the project a real production-style database.

Why AWS EC2 And Nginx

AWS EC2 was used for backend hosting because it gives full control over the Java runtime, systemd service, logs, and Nginx proxying. Nginx exposes the backend through port 80 and forwards API requests to Spring Boot on port 8080.

This deployment proves the backend can run as a real server process instead of only a local development app.

Why Vercel

Vercel was used for frontend hosting because it is fast, reliable, and simple for React/Vite deployment. The frontend uses a Vercel rewrite so /api calls are proxied to the EC2 backend.

Why Evidence-Backed AI

Memoris OS does not aim to be a generic chatbot. It answers from organization knowledge. The system retrieves authorized document chunks, sends only those chunks to the AI layer, and returns evidence cards with the final answer.

This makes the answer verifiable. Users can see which document or decision supported the response.

Summary Decision

The final architecture is:

```
Vercel React frontend
-> Vercel /api rewrite
-> AWS EC2 Nginx
-> Java Spring Boot backend
-> Neon PostgreSQL + pgvector
-> Gemini/OpenAI-ready AI service
```

This stack was selected because it balances hackathon speed, real backend credibility, secure enterprise workflow, and a practical path to production.